# Comprehensive Technical Overview: HarnessAgent

## 1. Business Problem

Deploying AI agents beyond simple chat demos into production environments introduces significant operational challenges. While agent prototypes are easy to build, they often fail when scaling due to:

- **Unpredictable Costs:** Unbounded LLM calls and excessive context retrieval (e.g., naive vector search) can lead to skyrocketing token spend.
- **Reliability & Loops:** Agents encountering API timeouts or unexpected inputs often enter infinite execution loops, exhausting budgets.
- **Safety & Compliance Risks:** Without strict guardrails, agents might execute destructive commands, hallucinate schemas, or leak Personally Identifiable Information (PII) in their responses.
- **Context Loss:** Managing short-term conversational context, long-term semantic knowledge, and structured data relationships across multiple agent steps is complex.
- **Maintenance Overhead:** When agent prompts fail to handle edge cases, manual debugging and prompt engineering are slow and error-prone.

**HarnessAgent** solves these challenges by providing a production-grade multi-agent execution platform. It acts as a control plane that wraps LLM agents with resilient routing, multi-tier memory, strict safety guardrails, and an automated self-improvement loop.

---

## 2. Architecture

HarnessAgent is designed with a service-oriented architecture, encapsulating agent execution into distinct, scalable layers.

### High-Level System Context (C4 Level 1)

- **Client Layer:** Developers and End Users interact with the system via web dashboards, CLIs, or direct REST API calls.
- **Harness Core:** The central execution environment (HarnessAgent) that manages the run loop, memory, and tools.
- **External Dependencies:**
  - **LLM APIs:** Anthropic, OpenAI, or local instances via vLLM/llama.cpp.
  - **Target Databases:** Postgres, SQLite, etc. (for the SQL Agent).
  - **MCP Servers:** Connect to filesystems, browsers, or custom APIs via the Model Context Protocol.

### Core Architecture Modules

- **API & Orchestration:** A FastAPI server handles inbound requests, enqueues jobs via Redis, and streams execution steps via Server-Sent Events (SSE).
- **Agent Worker (RQ):** Asynchronous workers dequeue jobs, spin up isolated environments, and execute the agent's multi-stage reasoning loop.
- **Memory System:** A three-tier architecture (Hot Redis for short-term, Warm Vector DB for semantics, and Structured Graph DB for knowledge).
- **Safety Pipeline:** A modular, three-stage guardrail system (Input, Intermediate, Output).
- **Hermes Self-Improvement Loop:** A background process that samples errors, proposes prompt patches, evaluates them, and applies them automatically.

---

## 3. Tech Stack

The platform is built on a modern, async-first Python stack with robust infrastructure services managed via Docker Compose:

| Layer | Technology | Purpose |
|---|---|---|
| **API Server** | FastAPI, uvicorn, Pydantic | REST endpoints, JSON schema validation, async request handling, SSE streaming. |
| **Worker Queue** | RQ (Redis Queue) | Background job processing and distributed worker execution. |
| **Short-Term Memory** | Redis | Conversation history, pub/sub for events, task queues, and circuit breaker states. |
| **Long-Term Memory** | Qdrant / ChromaDB | Semantic vector search for error patterns and document retrieval. |
| **Knowledge Graph** | Neo4j / NetworkX | Structured data representation and BFS traversal for Graph RAG. |
| **Observability** | MLflow, OpenTelemetry, Prometheus, Grafana | Agent tracing, metric scraping, and pre-built operational dashboards. |
| **LLM Integration** | Anthropic & OpenAI SDKs | Asynchronous completion calls, function calling, and provider failover. |

---

## 4. Agent Execution Flow

Every request processed by HarnessAgent follows a strict, trackable eight-stage execution flow. Here is a trace of a typical agent run:

1. **Request Ingestion:** The client calls `POST /runs`. FastAPI validates the payload, creates a `RunRecord` in Redis, and enqueues a job.
2. **Worker Activation:** An RQ Worker dequeues the job and initializes an `AgentRunner`, isolating the execution workspace.
3. **History & Context Fit:** The `ContextWindowManager` fits the conversation history into the token budget (e.g., using sliding windows or LLM summarization).
4. **Graph RAG Retrieval:** The `MemoryManager` extracts entities from the user prompt, anchors them in the Neo4j Knowledge Graph, performs a BFS traversal, and returns a highly compact `[SCHEMA]` block.
5. **Safety Check (Input/Intermediate):** The `Safety Pipeline` checks the request against injection attacks, loop detection thresholds, and budget limits.
6. **LLM Routing:** The `LLMRouter` checks provider health, respects `CircuitBreakers`, and calls the optimal model (e.g., falling back from Claude 3.5 Sonnet to GPT-4o-mini on failure).
7. **Tool Execution:** If the LLM requests a tool (e.g., `execute_sql`), the `ToolRegistry` validates the arguments, runs safety checks, executes the tool with a timeout, and writes the result back to short-term memory.

8. **Output Safety & Delivery:** Once the LLM formulates a final answer, the `PIIRedactor` strips sensitive data, MLflow closes the execution span, and the result is returned to the client (or streamed via SSE).

---

## 5. Services and Components

HarnessAgent is composed of several specialized, decoupled services:

- **LLM Router:** Acts as an air-traffic controller for LLMs. It manages context windows and provider health, automatically failing over using the `CircuitBreakerRegistry`.
- **Memory Manager & Graph RAG Engine:** Orchestrates Hot (Redis), Warm (Vector), and Structured (Graph) memory. The Graph RAG Engine reduces token usage by rendering compact graph paths instead of dumping raw documents.
- **Safety Pipeline:** A composable guardrail system featuring `InjectionDetector` (input), `LoopDetector` and `ToolPolicy` (intermediate), and `PIIRedactor` (output).
- **Tool Registry & MCP Client:** Validates and executes agent tools. The MCP Client auto-discovers external tools from Model Context Protocol servers over stdio or SSE.
- **Cost Tracker:** Intercepts every LLM call, calculates the USD cost using a predefined pricing table, and enforces monthly tenant budgets via Redis counters.
- **Hermes Worker:** A self-improvement daemon scheduled by `APScheduler`. It aggregates failed runs, uses an LLM to generate prompt patches, evaluates the patches against test cases, and automatically applies successful patches.

---

## 6. Impact, Value, & ROI

Implementing HarnessAgent transforms experimental AI projects into highly governed, measurable enterprise services.

- **83% Token Reduction (Graph RAG):** By retrieving specific graph relationships (tables, columns, foreign keys) rather than naive vector chunks, the LLM context is drastically compressed. This slashes per-request costs and lowers latency.
- **Guaranteed Reliability:** The Circuit Breaker pattern prevents cascading failures when third-party LLM providers experience outages, ensuring uninterrupted service by routing to healthy alternatives.
- **Strict Cost Control:** The Cost Tracker provides hard budget caps per tenant. Rogue agents stuck in loops are terminated by the Budget Guardrail before they incur massive API bills.
- **Automated Self-Improvement:** The Hermes loop reduces manual maintenance. If an agent consistently fails on specific edge cases, Hermes automatically writes, tests, and deploys a prompt patch to fix the behavior.
- **Enterprise-Grade Observability:** With MLflow, OpenTelemetry, and Grafana, engineering and risk teams have full visibility into token spend, cache hit rates, guardrail blocks, and tool failures.

*HarnessAgent connects engineering velocity to robust risk controls and financial discipline.*